

ADR-001: Arquitetura do 4Frames — processamento assíncrono de vídeos com upload direto ao S3, fila SQS e workers elásticos

Status

Pendente

Data

2026-08-19

1. Contexto

A FIAP X apresentou a investidores um projeto base que recebe um vídeo, extrai seus frames e devolve um **.zip** — um processo único e síncrono, sem autenticação, persistência ou fila. Os investidores querem transformá-lo em um produto em que vários usuários enviam vídeos e baixam o resultado.

O desafio exige processar vários vídeos ao mesmo tempo, não perder requisições em picos, proteger o acesso por usuário e senha, listar o status dos vídeos de cada usuário, notificar o usuário em caso de erro e ter persistência, escalabilidade, testes, CI/CD e monitoramento.

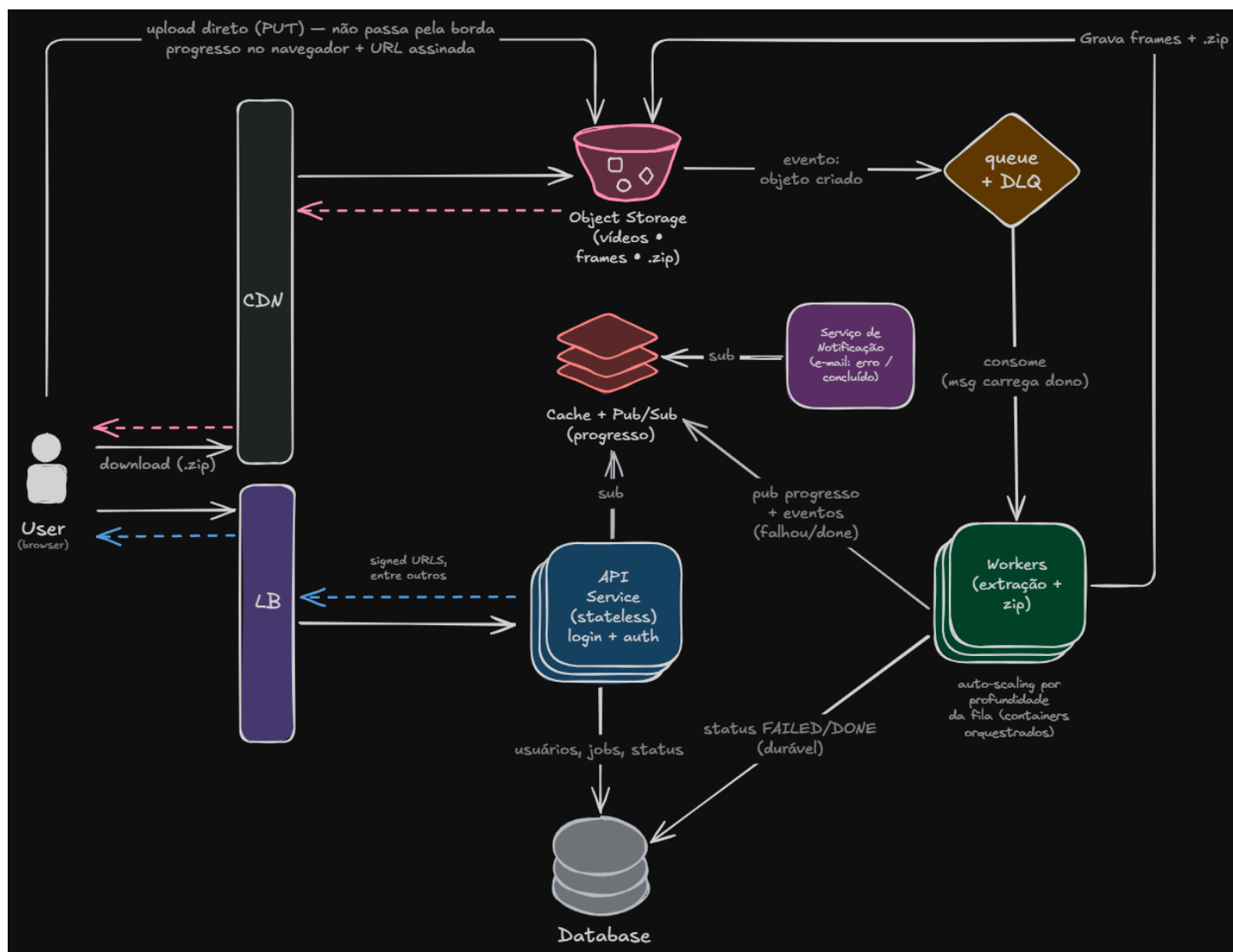
Duas características da carga orientam o desenho: o processamento de vídeo é longo e intensivo em CPU e I/O — inadequado ao modelo requisição/resposta — e o grosso dos bytes está no upload do vídeo e no download do **.zip**, não na lógica de negócio. A demanda é irregular, com picos.

Este ADR registra a arquitetura adotada pelo grupo para o 4Frames. A stack reaproveita o que o grupo já domina do Garagio — Node.js + TypeScript, Express, Prisma/PostgreSQL, SQS, EKS, Terraform, GitHub Actions e Datadog — em ambiente AWS.

2. Decisão

2.1. Visão geral

Arquitetura orientada a eventos com três planos escalados de forma independente: **controle** (API stateless: autenticação, jobs, URLs assinadas), **dados** (upload direto ao S3 e download pela CDN — os bytes não passam pela API) e **processamento** (workers consumindo uma fila SQS).



Componente	Responsabilidade	Tecnologia
LB	Entrada HTTP da API; distribui entre réplicas; health checks.	ALB (AWS Load Balancer Controller no EKS)
API Service (4frames-api)	Login/JWT; criação e listagem de jobs; URLs pré-assinadas de upload e download; progresso via SSE. Stateless, N réplicas.	Node.js 24 + TypeScript, Express 5, Zod, Prisma; container no EKS com HPA
Object Storage	Vídeos, frames e .zip em bucket privado; emite evento de objeto criado.	Amazon S3 (presigned URLs + Event Notifications)
fila + DLQ	Retém os eventos de upload até haver worker; isola mensagens envenenadas.	Amazon SQS (fila padrão + DLQ por redrive policy)
Workers (4frames-worker)	Consomem a fila, extraem frames, geram o .zip, gravam no S3, publicam progresso e gravam o status final.	Node.js + TypeScript + ffmpeg; container no EKS escalado pelo KEDA (scaler aws-sqs-queue)

Cache + Pub/Sub	Progresso efêmero (cache com TTL) e canal de eventos do job.	Amazon ElastiCache for Redis (ioredis)
Serviço de Notificação (4frames-notifier)	Assina eventos de falha e conclusão e envia e-mail ao dono do job.	Node.js + TypeScript, nodemailer + templates Pug, Amazon SES (SMTP)
Database	Usuários, jobs e status durável — fonte de verdade da listagem.	Amazon RDS PostgreSQL 16 + Prisma
CDN	Borda para download do .zip, com origem no bucket privado.	Amazon CloudFront (OAC + URL assinada)

Em ambiente local o mesmo código roda com Docker Compose — PostgreSQL, LocalStack (S3 + SQS), Redis e Mailpit —, padrão já usado no Garagio.

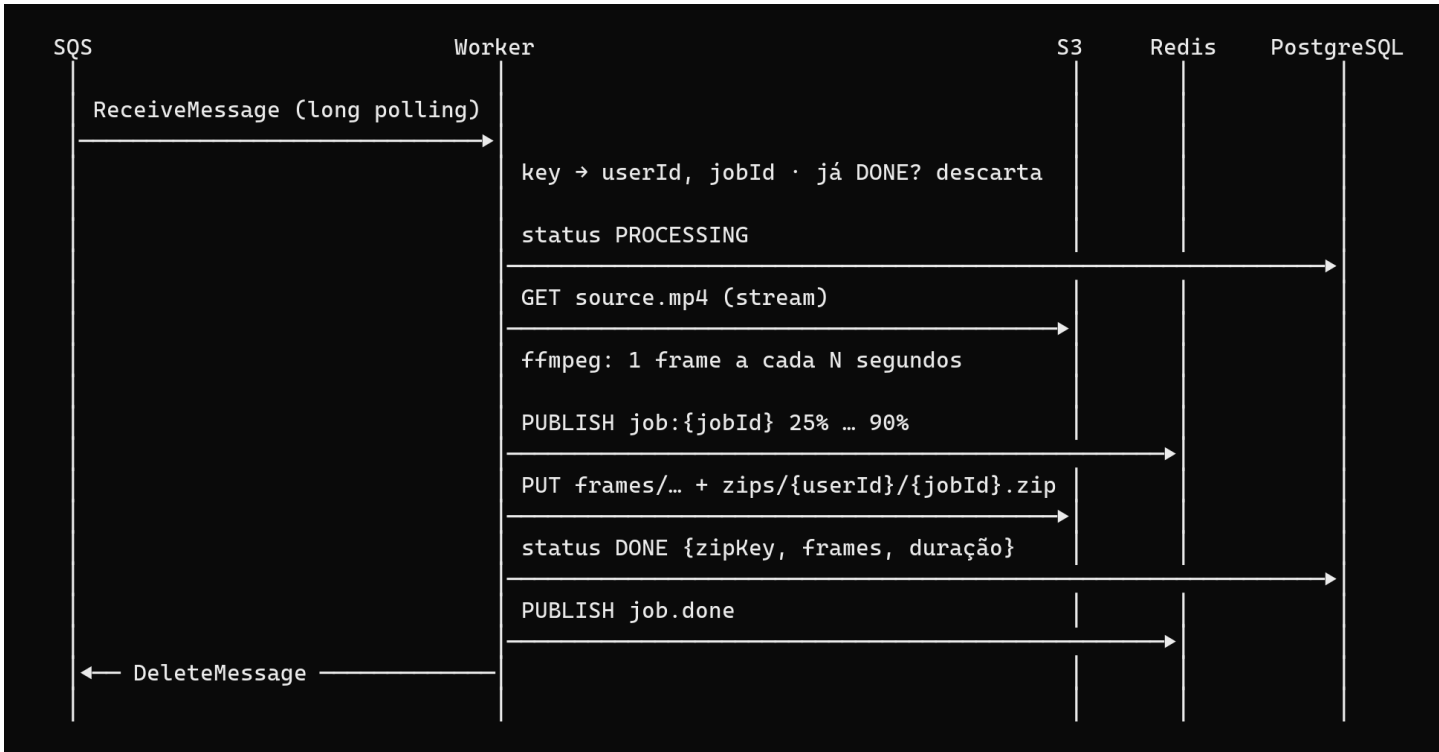
2.2. Fluxo ponta a ponta

1. **Login** (POST /auth/login, usuário + senha com hash bcrypt) devolve um JWT, exigido nas demais rotas.
2. **Solicitação de upload** (POST /videos com nome, tamanho e tipo): a API valida limites, grava o job como `UPLOAD_PENDING` e devolve uma URL pré-assinada de PUT para `videos/{userId}/{jobId}/source.{ext}`, de validade curta e restrita a chave, método e content-type.
3. **Upload direto**: o navegador faz o PUT no S3 — sem passar por LB, API ou CDN — e exibe o progresso localmente. Ao terminar chama POST /videos/{jobId}/complete; a API confirma o objeto (HEAD) e marca `QUEUED`.
4. **Evento**: o S3 emite *ObjectCreated* (filtro por prefixo `videos/`) para a fila SQS. A chave do objeto identifica dono e job — o worker não consulta a API.
5. **Processamento**: o worker recebe a mensagem (long polling; visibility timeout maior que o processamento, renovado durante o job); se o job já estiver `DONE`, descarta. Marca `PROCESSING`, baixa o vídeo em streaming, extrai 1 frame a cada N segundos com ffmpeg e publica o progresso em `job:{jobId}` (Redis), gravando o último percentual em `progress:{jobId}` com TTL.
6. **Resultado**: grava frames e .zip em chaves determinísticas (`zips/{userId}/{jobId}.zip`), depois grava `DONE` (chave do zip, nº de frames, duração) no PostgreSQL, publica `job.done` e apaga a mensagem. Vídeo inválido → `FAILED` + `job.failed`. Falha transiente → a mensagem volta à fila e, após `maxReceiveCount`, vai à DLQ (job `FAILED` + alerta).
7. **Notificação**: o notificador assina `job.failed` e `job.done` e envia e-mail ao dono com o motivo da falha ou o link de download.
8. **Acompanhamento e download**: GET /videos lista os jobs do usuário (status do PostgreSQL + percentual do Redis); GET /videos/{jobId}/events retransmite o canal do job via SSE; GET /videos/{jobId}/download (job `DONE` do próprio usuário) devolve uma URL assinada do CloudFront, cuja origem é o bucket privado.

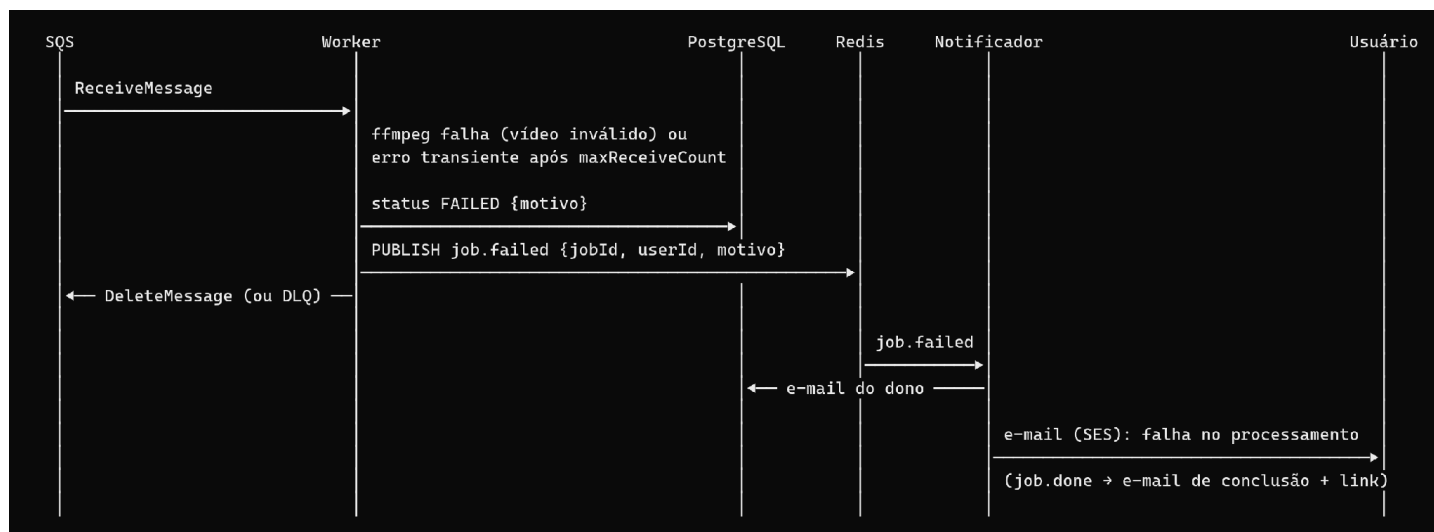
Upload e enfileiramento



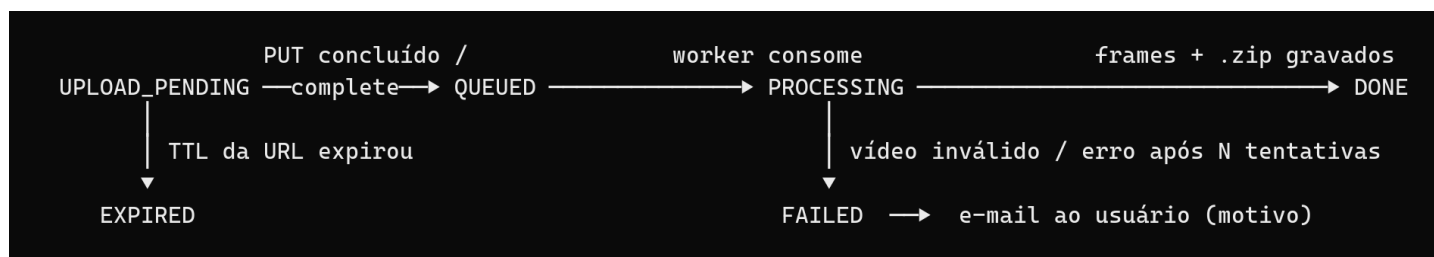
Processamento — sucesso



Falha e notificação



Estados do job



2.3. Garantias operacionais

Idempotência O SQS entrega *at-least-once*; o worker checa o status antes de processar e grava em chaves determinísticas — uma duplicata, no pior caso, sobrescreve o mesmo **.zip**.

Ordem de gravação Artefato no S3 → status no banco → evento no Redis. Quem recebe a notificação encontra artefato e status coerentes; o progresso é efêmero e sua perda custa apenas o percentual.

Sem laço de eventos A notificação do bucket filtra o prefixo **videos/**; as gravações dos workers em **frames/** e **zips/** não geram mensagens.

Auto-scaling O KEDA escala os workers pela métrica **ApproximateNumberOfMessagesVisible** (mínimo 0–1, máximo N); **terminationGracePeriodSeconds** cobre a duração de um job para o scale-down não interromper processamentos.

Uploads abandonados Jobs **UPLOAD_PENDING** além da validade da URL viram **EXPIRED** por rotina periódica; regras de ciclo de vida do bucket removem objetos órfãos e artefatos antigos.

Segurança JWT em todas as rotas e autorização por dono; URLs pré-assinadas com escopo e validade curta; bucket privado servido pelo CloudFront via OAC; credenciais por IRSA com menor privilégio; segredos no Kubernetes.

Observabilidade OpenTelemetry (OTLP) → Datadog, logs pino correlacionados por `jobId`, CloudWatch para pods e métricas do SQS; alertas para DLQ não vazia e jobs presos em `PROCESSING`.

2.4. Qualidade e entrega

- Um repositório por serviço (`4frames-api`, `4frames-worker`, `4frames-notifier`) e um de infraestrutura (`4frames-infra`, Terraform: VPC, EKS, ECR, ALB, S3, SQS, CloudFront, ElastiCache, RDS) — mesmo arranjo do Garagio.
- Arquitetura hexagonal/DDD; pnpm, ESLint, Prettier e Husky.
- Testes: Jest + ts-jest + supertest (unitários e de API); integração com LocalStack e PostgreSQL no Compose; vídeos de amostra no worker (contagem de frames e conteúdo do zip); k6 para carga.
- GitHub Actions: test → lint → SonarQube → build e push no ECR → deploy no EKS (staging por branch; produção por tag `release-*`).

2.5. Atendimento aos requisitos

Requisito	Como é atendido
Vários vídeos ao mesmo tempo	N workers em paralelo; réplicas seguem a profundidade da fila (KEDA).
Não perder requisição em picos	Upload direto ao S3; evento retido no SQS até haver worker; API sem bytes nem estado.
Usuário e senha	Login com bcrypt + JWT; autorização por dono em todas as rotas.
Listagem de status por usuário	Jobs no PostgreSQL + progresso no Redis; atualização via SSE.
Notificação em caso de erro	<code>job.failed</code> → e-mail via SES (também em conclusão).
Persistência e escalabilidade	RDS + S3; API com HPA; workers com KEDA; CloudFront na borda.
Testes, CI/CD e monitoramento	Jest/supertest/k6; GitHub Actions + Terraform; OpenTelemetry/Datadog.

3. Decisões Arquiteturais

Decisão	Justificativa / Racional
Upload direto ao S3 com URL pré-assinada	LB e API ficam fora do caminho dos bytes e a banda escala com o S3; o navegador mostra o progresso; URL com escopo e validade curta mantém o bucket privado.
Upload fora da CDN; download pela CDN	Upload é escrita única e não cacheável; o download é um artefato estático — o cache reduz latência e egress.

Processamento disparado por evento S3 → SQS	Só começa com os bytes persistidos, independe de o cliente continuar conectado e nunca há job na fila sem arquivo; o filtro por prefixo evita laço.
SQS padrão + DLQ	Absorve picos sem perder requisição; retries nativos por visibility timeout; envenenadas isoladas por redrive.
Chave do objeto codifica dono e job	Worker autônomo, sem chamada à API no caminho crítico.
Workers no EKS escalados pelo KEDA	A fila mede o backlog diretamente (CPU reage tarde); permite scale-to-zero fora de pico.
API stateless atrás do ALB	Escala horizontal; JWT dispensa sessão; qualquer réplica atende qualquer requisição, inclusive SSE via Pub/Sub.
Redis para progresso; PostgreSQL para status	Progresso é efêmero e de alta frequência; o banco guarda só transições e é a fonte de verdade.
Notificador como assinante desacoplado	E-mail fora do caminho crítico; falha do provedor não afeta o processamento.
Reuso da stack do Garagio	Node/TS, Express, Prisma, consumer SQS, auth JWT, e-mail (nodemailer/Pug), OpenTelemetry/Datadog, Terraform e pipelines já prontos — menor risco e curva de aprendizado.

4. Consequências

4.1. Positivas

- Paralelismo e resiliência a picos vêm da fila e dos workers elásticos — a API nunca é gargalo, pois não transporta bytes nem guarda estado.
- Custo proporcional à demanda: workers escalam com a fila (inclusive a zero); CloudFront reduz o egress do S3.
- Falhas isoladas: worker cai → mensagem volta à fila; Redis cai → perde só o percentual; notificador cai → e-mails atrasam.

4.2. Negativas e Trade-offs

- Mais peças para operar: S3, SQS/DLQ, Redis, CloudFront, ALB, RDS, KEDA e notificador.
- Consistência eventual entre upload, consumo e status; progresso best-effort.
- Redis Pub/Sub é *at-most-once*: notificador fora do ar perde o evento — mitigado por flag `notified_at` no job e varredura periódica (ou eventos terminais via SNS → SQS).
- Vídeo inválido só é detectado no worker; uploads abandonados exigem expiração e ciclo de vida no bucket.
- Um vídeo por worker por vez; vídeos longos ocupam o worker por minutos — limites de tamanho e duração no upload.

- Conta AWS Academy: LabRole (sem roles próprias para IRSA/KEDA) e SES em sandbox — usar LabRole e SMTP de testes quando necessário.
-

5. Compatibilidade

- Mesmo resultado do projeto base (`.zip` com os frames, mesma regra de extração), agora gerado pelo worker.
- O contrato passa a ser assíncrono: `jobId`, acompanhamento de status e download quando `DONE`.
- Toda rota exige JWT; limites de formato e tamanho são validados no upload e no worker.